

4.1 Minimizing the Sum of Completion Times on a Single Machine

Presenter: Tanvir Hossain
Student Id: 2005014

December 8, 2025

Outline

- 1 Problem Definition
- 2 The Challenge & Strategy
- 3 Simulation: Preemptive Scheduling
- 4 2-Approximation Algorithm
- 5 Simulation: Non-Preemptive Scheduling
- 6 Observation 4.1
- 7 Lemma 4.2: Bounding the Delay
- 8 Theorem 4.3: The Result
- 9 Summary

The Problem Statement

Input: n jobs, each with:

Constraints:

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)

Constraints:

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)
- Release date $r_j \geq 0$ (integer)

Constraints:

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)
- Release date $r_j \geq 0$ (integer)

Constraints:

- Single machine (process one job at a time)

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)
- Release date $r_j \geq 0$ (integer)

Constraints:

- Single machine (process one job at a time)
- No job processed before its release date

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)
- Release date $r_j \geq 0$ (integer)

Constraints:

- Single machine (process one job at a time)
- No job processed before its release date
- Jobs must be processed **nonpreemptively**

The Problem Statement

Input: n jobs, each with:

- Processing time $p_j > 0$ (integer)
- Release date $r_j \geq 0$ (integer)

Constraints:

- Single machine (process one job at a time)
- No job processed before its release date
- Jobs must be processed **nonpreemptively**

Goal: Minimize $\sum_{j=1}^n C_j$ where C_j is the completion time of job j

Nonpreemptive vs Preemptive Scheduling

Nonpreemptive

Once a job starts, it must complete without interruption

Preemptive

Jobs can be interrupted and resumed later

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.
- This is an **NP-Hard** problem.

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.
- This is an **NP-Hard** problem.

The Strategy

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.
- This is an **NP-Hard** problem.

The Strategy

- 1 **Relaxation:** Solve the easier **Preemptive** version first (SRPT Rule).

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.
- This is an **NP-Hard** problem.

The Strategy

- 1 **Relaxation:** Solve the easier **Preemptive** version first (SRPT Rule).
- 2 **Rounding:** Use the completion order from the preemptive schedule.

The Challenge & Strategy

Why is this hard?

- We cannot stop a job once it starts.
- If a short job arrives while we are processing a long job, the short job has to wait.
- This is an **NP-Hard** problem.

The Strategy

- 1 **Relaxation:** Solve the easier **Preemptive** version first (SRPT Rule).
- 2 **Rounding:** Use the completion order from the preemptive schedule.
- 3 **Construction:** Schedule non-preemptively based on that order.

Shortest Remaining Processing Time (SRPT)

Algorithm:

- ① Start at time 0
- ② At each time, schedule the job with **smallest remaining processing time** among:
 - Jobs that have been released ($t \geq r_j$)
 - Jobs not yet completed
- ③ Continue until:
 - Job completes, OR
 - New job arrives
- ④ Repeat

Shortest Remaining Processing Time (SRPT)

Algorithm:

- ① Start at time 0
- ② At each time, schedule the job with **smallest remaining processing time** among:
 - Jobs that have been released ($t \geq r_j$)
 - Jobs not yet completed
- ③ Continue until:
 - Job completes, OR
 - New job arrives
- ④ Repeat

Key Property

SRPT finds the **optimal preemptive schedule** in polynomial time!

Simulation Example

Consider these 3 Jobs. We will track them through the simulation.

Job	Release (r_j)	Duration (p_j)	Visual
Job 1	0	2	
Job 2	1	5	
Job 3	3	1	

Step 1: Let's run the **Preemptive** Simulation (SRPT).

Simulation: Optimal Preemptive (SRPT)

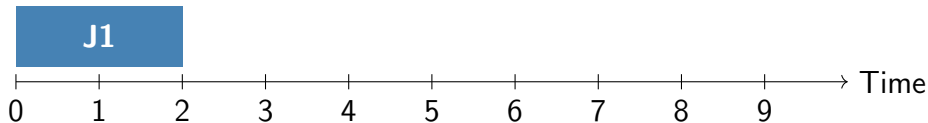
Rule: Shortest Remaining Processing Time.



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1.



Simulation: Optimal Preemptive (SRPT)

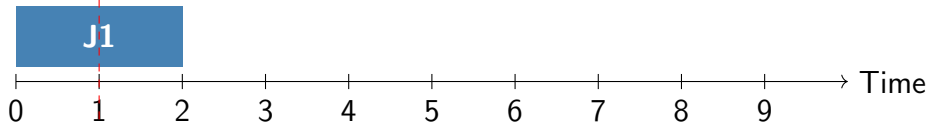
Rule: Shortest Remaining Processing Time.

$t = 1$: J2 arrives ($p = 5$).

J1 rem = 1.

$1 < 5$, so **Continue J1**.

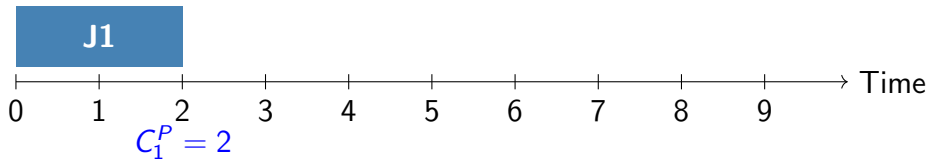
$t = 0$: J1 arrives ($p = 2$). Run J1.



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1.



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

INTERRUPT!
 $t = 3$: J3 arrives ($p = 1$).
J2 rem = 4 ($5 - 1$).
 $1 < 4$, so **Switch to J3**.

$t = 0$: J1 arrives ($p = 2$). Run J1.

$t = 2$: J1 done. Run J2 ($p = 5$).



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).



Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).

$t = 4$: J3 done. Resume J2.

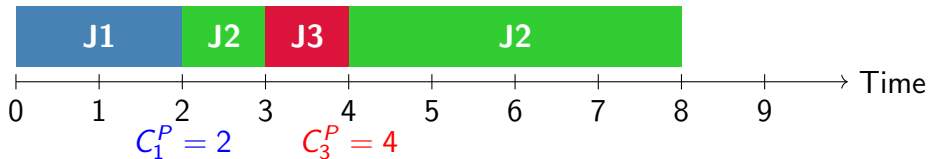


Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).

$t = 4$: J3 done. Resume J2.

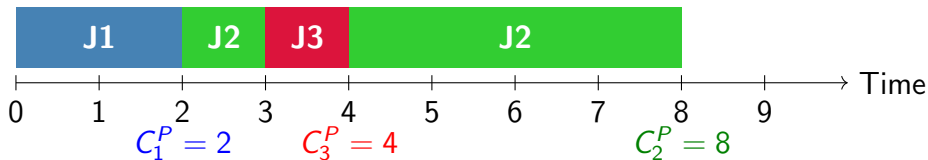


Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).

$t = 4$: J3 done. Resume J2.

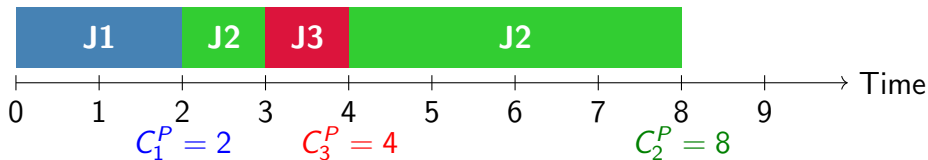


Simulation: Optimal Preemptive (SRPT)

Rule: Shortest Remaining Processing Time.

$t = 0$: J1 arrives ($p = 2$). Run J1. $t = 2$: J1 done. Run J2 ($p = 5$).

$t = 4$: J3 done. Resume J2.



Total Completion Time: $\sum_j C_j = 2 + 4 + 8 = \boxed{14}$

Final Preemptive Order: Job 1 $\rightarrow$ Job 3 $\rightarrow$ Job 2.

2-Approximation Algorithm for nonpreemptive scheduling

Algorithm

- ① Find optimal preemptive schedule using SRPT
- ② Let $C_1^P \leq C_2^P \leq \dots \leq C_n^P$ (reindex jobs by completion order)
- ③ Schedule jobs **nonpreemptively** in this order:
 - Job 1: from r_1 to $r_1 + p_1$
 - Job 2: from $\max(r_1 + p_1, r_2)$ to $\max(r_1 + p_1, r_2) + p_2$
 - Job j : from $\max\{C_{j-1}^N, r_j\}$ to $\max\{C_{j-1}^N, r_j\} + p_j$

2-Approximation Algorithm for nonpreemptive scheduling

Algorithm

- ① Find optimal preemptive schedule using SRPT
- ② Let $C_1^P \leq C_2^P \leq \dots \leq C_n^P$ (reindex jobs by completion order)
- ③ Schedule jobs **nonpreemptively** in this order:
 - Job 1: from r_1 to $r_1 + p_1$
 - Job 2: from $\max(r_1 + p_1, r_2)$ to $\max(r_1 + p_1, r_2) + p_2$
 - Job j : from $\max\{C_{j-1}^N, r_j\}$ to $\max\{C_{j-1}^N, r_j\} + p_j$

Intuition: Jobs that finish early in preemptive schedule should finish early in nonpreemptive schedule too!

2-Approximation Algorithm (contd.)

Now we apply this algorithm to construct the real schedule.

Algorithm

- 1 Sort jobs by their Preemptive Completion times (C_j^P).

2-Approximation Algorithm (contd.)

Now we apply this algorithm to construct the real schedule.

Algorithm

- 1 Sort jobs by their Preemptive Completion times (C_j^P).
- 2 Order obtained: **1, 3, 2**.

2-Approximation Algorithm (contd.)

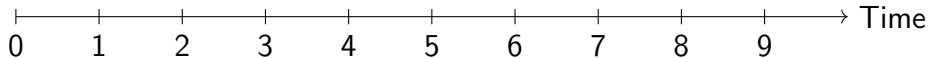
Now we apply this algorithm to construct the real schedule.

Algorithm

- 1 Sort jobs by their Preemptive Completion times (C_j^P).
- 2 Order obtained: **1, 3, 2**.
- 3 Schedule them in this **fixed order** non-preemptively.

Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.



Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

Run J1 ($r_1 = 0$)

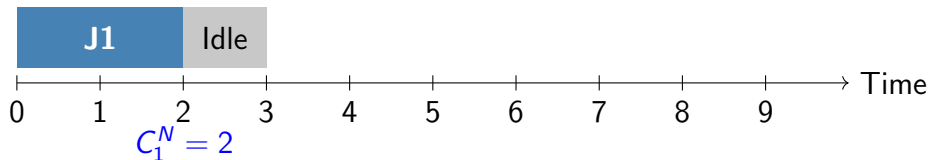


Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

Run J1 ($r_1 = 0$)

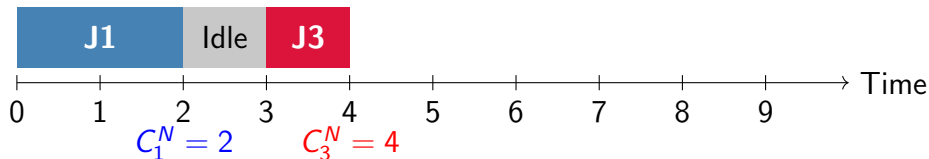
Next is J3. But $r_3 = 3$. Wait!



Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

Run J1 ($r_1 = 0$)

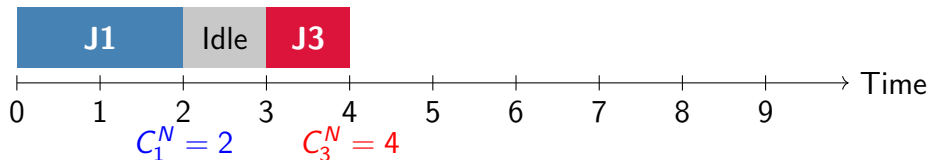


Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

Run J1 ($r_1 = 0$)

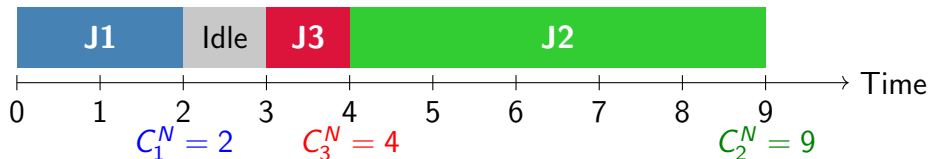
Next is J2 ($r_2 = 1$). Ready to go!



Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

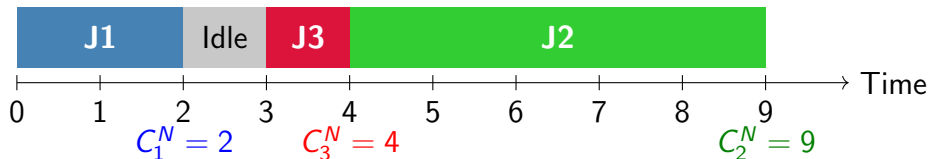
Run J1 ($r_1 = 0$)



Simulation: Non-Preemptive Construction

Fixed Order: $1 \rightarrow 3 \rightarrow 2$.

Run J1 ($r_1 = 0$)



Total Completion Time: $2 + 4 + 9 = \boxed{15}$.

Observation 4.1

Observation 4.1

$$\sum_{j=1}^n C_j^P \leq \text{OPT}$$

where C_j^P is completion time in optimal preemptive schedule and OPT is optimal nonpreemptive objective.

Proof:

Observation 4.1

Observation 4.1

$$\sum_{j=1}^n C_j^P \leq \text{OPT}$$

where C_j^P is completion time in optimal preemptive schedule and OPT is optimal nonpreemptive objective.

Proof:

- Any nonpreemptive schedule is also a valid preemptive schedule

Observation 4.1

Observation 4.1

$$\sum_{j=1}^n C_j^P \leq \text{OPT}$$

where C_j^P is completion time in optimal preemptive schedule and OPT is optimal nonpreemptive objective.

Proof:

- Any nonpreemptive schedule is also a valid preemptive schedule
- The optimal preemptive schedule is at least as good as any preemptive schedule

Observation 4.1

Observation 4.1

$$\sum_{j=1}^n C_j^P \leq \text{OPT}$$

where C_j^P is completion time in optimal preemptive schedule and OPT is optimal nonpreemptive objective.

Proof:

- Any nonpreemptive schedule is also a valid preemptive schedule
- The optimal preemptive schedule is at least as good as any preemptive schedule
- Therefore, optimal preemptive $\leq$ optimal nonpreemptive

Lemma 4.2: Bounding the Delay

Lemma 4.2

For each job $j = 1, \dots, n$: $C_j^N \leq 2C_j^P$

Here jobs are indexed according to the ascending order of their completion time.

Lemma 4.2: Bounding the Delay

Lemma 4.2

For each job $j = 1, \dots, n$: $C_j^N \leq 2C_j^P$

Here jobs are indexed according to the ascending order of their completion time.

Proof Idea: We'll show C_j^N cannot be too large by using two lower bounds on C_j^P .

Lower Bounds on C_j^P :

Lemma 4.2: Bounding the Delay

Lemma 4.2

For each job $j = 1, \dots, n$: $C_j^N \leq 2C_j^P$

Here jobs are indexed according to the ascending order of their completion time.

Proof Idea: We'll show C_j^N cannot be too large by using two lower bounds on C_j^P .

Lower Bounds on C_j^P :

① **Release dates:** Job j cannot complete before any of jobs $1, \dots, j$ are released:

$$C_j^P \geq \max_{k=1, \dots, j} r_k$$

Lemma 4.2: Bounding the Delay

Lemma 4.2

For each job $j = 1, \dots, n$: $C_j^N \leq 2C_j^P$

Here jobs are indexed according to the ascending order of their completion time.

Proof Idea: We'll show C_j^N cannot be too large by using two lower bounds on C_j^P .

Lower Bounds on C_j^P :

① **Release dates:** Job j cannot complete before any of jobs $1, \dots, j$ are released:

$$C_j^P \geq \max_{k=1, \dots, j} r_k$$

② **Processing times:** Jobs $1, \dots, j$ need time to process:

$$C_j^P \geq \sum_{k=1}^j p_k$$

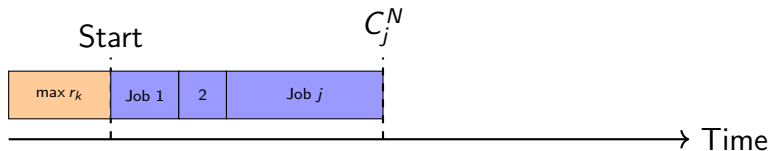
Lemma 4.2: Upper Bound on C_j^N

Key Observation: In nonpreemptive schedule, machine is idle only when waiting for next job's release.

Therefore: $C_j^N \leq \max_{k=1, \dots, j} r_k + \sum_{k=1}^j p_k$

Lemma 4.2: Upper Bound on C_j^N

Key Observation: In nonpreemptive schedule, machine is idle only when waiting for next job's release.



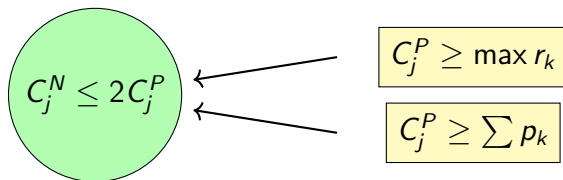
No idle time in this interval!
Length $\leq \sum_{k=1}^j p_k$

Therefore: $C_j^N \leq \max_{k=1, \dots, j} r_k + \sum_{k=1}^j p_k$

Lemma 4.2: Completing the Proof

We have:

$$\begin{aligned} C_j^N &\leq \max_{k=1,\dots,j} r_k + \sum_{k=1}^j p_k \\ &\leq C_j^P + C_j^P \quad (\text{using both lower bounds}) \\ &= 2C_j^P \end{aligned}$$



onslidej2-**Result:** Each job's completion time at most doubles!

Theorem 4.3: The Result

Theorem 4.3

Scheduling in order of the completion times of an optimal preemptive schedule is a **2-approximation algorithm** for scheduling a single machine with release dates to minimize the sum of completion times.

Proof:

Theorem 4.3: The Result

Theorem 4.3

Scheduling in order of the completion times of an optimal preemptive schedule is a **2-approximation algorithm** for scheduling a single machine with release dates to minimize the sum of completion times.

Proof:

$$\begin{aligned}\sum_{j=1}^n C_j^N &\leq 2 \sum_{j=1}^n C_j^P \quad (\text{by Lemma 4.2}) \\ &\leq 2 \cdot \text{OPT} \quad (\text{by Observation 4.1})\end{aligned}$$

Theorem 4.3: The Result

Theorem 4.3

Scheduling in order of the completion times of an optimal preemptive schedule is a **2-approximation algorithm** for scheduling a single machine with release dates to minimize the sum of completion times.

Proof:

$$\begin{aligned}\sum_{j=1}^n C_j^N &\leq 2 \sum_{j=1}^n C_j^P \quad (\text{by Lemma 4.2}) \\ &\leq 2 \cdot \text{OPT} \quad (\text{by Observation 4.1})\end{aligned}$$

Our algorithm's cost $\leq 2 \times$ Optimal cost

Key Takeaways

- ❶ **Problem:** Schedule n jobs on single machine nonpreemptively to minimize $\sum C_j$

Key Takeaways

- ① **Problem:** Schedule n jobs on single machine nonpreemptively to minimize $\sum C_j$
- ② **SRPT Rule:** Optimal algorithm for preemptive version (always schedule job with shortest remaining processing time)

Key Takeaways

- ① **Problem:** Schedule n jobs on single machine nonpreemptively to minimize $\sum C_j$
- ② **SRPT Rule:** Optimal algorithm for preemptive version (always schedule job with shortest remaining processing time)
- ③ **Our Algorithm:**
 - Find optimal preemptive schedule
 - Schedule jobs nonpreemptively in same completion order

Key Takeaways

- ① **Problem:** Schedule n jobs on single machine nonpreemptively to minimize $\sum C_j$
- ② **SRPT Rule:** Optimal algorithm for preemptive version (always schedule job with shortest remaining processing time)
- ③ **Our Algorithm:**
 - Find optimal preemptive schedule
 - Schedule jobs nonpreemptively in same completion order
- ④ **Result:** 2-approximation algorithm
 - Uses two key bounds: release dates and total processing time
 - Each job's completion time at most doubles

Visual Summary

Preemptive (SRPT)



↓ Use completion order

Nonpreemptive



2-approx
guar-
anteed!

Any Questions?

Thank You!

Presenter: Tanvir Hossain
Student ID: 2005014